IETF 127 BoF proposal · ART Area

MTA Hooks

An HTTP protocol that lets an MTA delegate processing decisions to external services, across the whole life of a message.

draft-degennaro-mta-hooks-02
Mauro De Gennaro · Stalwart Labs

One state object
Session, envelope, message and delivery, described once.

One JSON Patch
Every decision is a change to that state.

Grants
The MTA decides what may change.

Nine stages
From connect to the DSN.

Plain HTTP
JSON or CBOR.

Before we start

The decision in front of this room

We are asking

- Is this problem real?
- Is it well scoped?
- Is it worth solving here?
- Will people do the work?

We are not asking

- Approve this wire format.
- Ratify a data model.
- Bless one implementation.

The draft is evidence that the problem is tractable. It is not the thing on the table today.

01

The problem

Mail servers delegate decisions to external services. That delegation covers the conversation that receives a message, and stops there.

Prior art

Milter, in one slide

`connect` `helo` `envfrom` `envrcpt` `header` `body` `eom`

Twenty five years of deployment

Sendmail, around 2000. A C library, `libmilter`. Callbacks along the SMTP conversation; the filter may reject, defer, discard or modify.

The ecosystem speaks it

Rspamd, ClamAV, SpamAssassin, OpenDKIM, OpenDMARC. It is the only portable way to filter mail today.

This is not an argument that Milter is bad. It is an argument about reach.

Reach

Milter's reach ends at DATA

connect	helo	mail	rcpt	data	route	transmit	deliver	dsn
hook	hook	hook	hook	hook	nothing	nothing	nothing	nothing

One protocol covers this

A different mechanism per MTA covers this

The moment the server answers `250 Ok`, the filter is out of the picture. It never learns whether what it accepted was delivered.

After acceptance

Everything after acceptance, today

What you need	How it is wired today
Choose a transport or next hop	Lookup tables: one key string in, one value out
Archive with proof it was archived	BCC maps, with no return channel
Know whether recipient B received it	VERP addressing, a bounce mailbox, and log parsing
Report on delivery TLS	A dedicated side channel with its own collector chain
Suppress a notification	Global switches, or nothing

None is an IETF specification. None is portable. None is correlated with the others, or with what was decided at reception.

Lookup tables

A routing decision gets one string

A transport lookup sees

one key

The recipient domain or address. One value comes back: a transport and next hop.

Invisible to the lookup

- The authenticated sender or the client that submitted the message
- The verdict of the spam filter at reception
- How many attempts failed, and how the last one failed
- Whether TLS failed on the default path

Failover after three failed attempts, or a VPN route that must hold or the mail waits, has no standard place to live.

Context

Context arrives by configuration, and silently does not

Macros

The client address, reverse name, TLS parameters and authenticated identity must each be exported by the administrator, per stage, by name.

A forgotten macro is not an error. The filter cannot tell "not configured" from "not available".

No negotiation

A filter cannot ask for what it needs, discover what the server can provide, or refuse to start when an input is missing.

No authentication results

SPF, DKIM and DMARC are not part of the interface. Every filter recomputes them or parses headers.

No session signals

Pregreet traffic, pipelining violations, command counts and an unverified client certificate stay invisible.

Extensibility

Adding one new field

A DKIM2 signer needed one datum from Postfix: proof that a DSN really came from the local bounce daemon.

Step 1

Patch the core

bounce(8) passes evidence to cleanup(8) through a private record.

Step 2

New macros

`{postfix_dsn_evidence}` and the original envelope.

Step 3

Document it

An update to MILTER_README so anyone can find it.

Step 4

Roll it out

Every filter author learns the names and requests them.

The real ceiling is not "can it carry X". It is what it costs to introduce X at all. (postfix-devel, August 2026)

Chains

Filters cannot talk to each other

The only channel is a header

A filter that wants to pass a score to the next one writes `X-Spam-Status` into the message. The next filter parses it.

- Unstructured, and forgeable by the sender
- Travels onward with the mail
- No way to learn who quarantined it, or why

No single call

A filter that wants the whole message with its metadata takes every callback and rebuilds it. The Rspamd proxy does exactly this, then makes one HTTP call.

A requirement arriving now

DKIM2 asks for things that happen after acceptance

Section 10.6

The signature records the `RCPT TO` values of the SMTP transaction that carried the copy. A `Bcc:` address **MUST NOT** appear in the copy sent to anyone else.

Section 16.1

A DSN **MUST** carry its own Message-Instance and DKIM2-Signature fields, and goes to the `mf=` address of the highest numbered signature.

Signing per SMTP transaction, not per queued message, and a hook at DSN generation.

draft-ietf-dkim-dkim2-spec-07

A requirement arriving now

One Milter cannot produce that signature

The filter runs early

At `cleanup`. One queued message, one body, one recipient list. The delivery job has not been split yet.

The split happens later

Later, in `qmgr`, when recipients are grouped by transport and next hop. No filter runs there.

At signing time the filter can put every recipient in the signature, disclosing the blind copies, or guess a recipient set that does not match the transaction. Bounces fare no better: there is no hook before a DSN is injected, so nothing can sign it.

A filter cannot consolidate its way to data the server has not computed yet.

Demand

The people asking for this

Not the filter vendors

Rspamd, SpamAssassin and ClamAV paid the integration cost once and amortise it across every deployment.

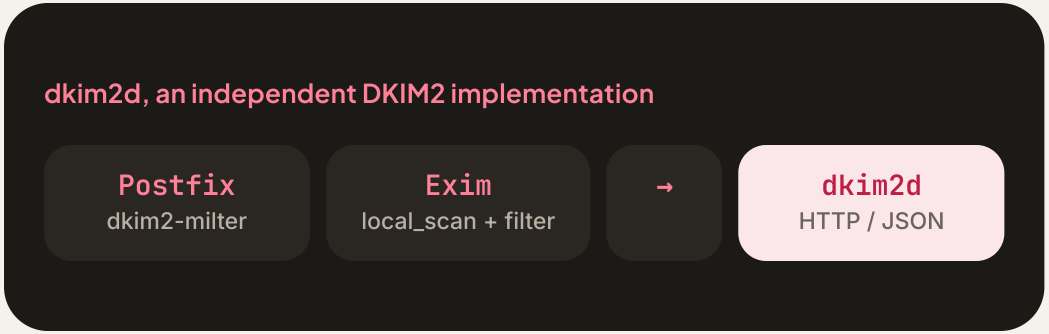
The operators

People who want one site-specific rule. Check the recipient against our directory. Apply the footer legal asked for. Push an event into our SIEM. Fifty lines, in whatever language the shop already uses.

Stalwart ships Milter support. Its users asked for an HTTP interface anyway: a revealed preference from people who had both options.

Demand

Scanners already want to be HTTP services



The adapter zoo

The logic lives in one HTTP daemon. Each MTA needs its own adapter to gather the envelope, call it, and validate what comes back.

The adapters are the missing standard, rebuilt per MTA from whatever interface each one offers.

An obvious alternative

Specifying Milter would not close the gap

Reach

A specification of the existing protocol still stops when the message is accepted.

Extensibility

Context still arrives by macro, and a new field still costs a core change in every server.

Audience

The operator with a fifty-line rule still cannot practically use it.

Outbound delegation is not an unimplemented Milter feature. Its whole vocabulary answers one question: what do I say to the client waiting on this command.

Summary

The problem, in short

Delegation covers the conversation that receives a message, and stops there.

- Everything after acceptance is a different mechanism in every server, none standard, none correlated
- A new field costs a change to the server core
- Requirements from IETF work, such as DKIM2, cannot be expressed at all
- The people who most need to write filters are the least able to

Is that the right problem, and is this the right place to solve it?

02

The idea

Shown as evidence the problem is tractable, not as a proposal to ratify.

The idea

Three ideas carry the whole design

1

One state object

At each stage the MTA describes the session, the envelope with every recipient's status, the message and the delivery attempts, in one object.

2

One JSON Patch

The scanner answers with an RFC 6902 patch against that object. The MTA carries out the resulting state.

3

The MTA controls change

Each scanner may change only the paths it was granted, and by default only toward stricter processing.

The idea

Every decision is a change to the state

Decision	Path it changes
Accept, reject, quarantine	`/action`, `/response`
Add a header field	`/message/headers`
Drop a recipient	`/envelope/to/*/status`
Choose a route	`/envelope/to/*/route`
Suppress a DSN	`/envelope/to/*/status`
Hand data to the next scanner	`/ext/org.example.spam`

No other way to influence processing exists.

So every effect a scanner can have is a path, and every permission is a grant on a path.

Architecture

The MTA is always the client

Optional

Discover

GET `/.well-known/mta-hooks`

Once

Register

POST stages, properties and grants

Per stage

Hook request

POST the current state

Per stage

JSON Patch

Validated against grants, then applied

The MTA exposes no HTTP endpoint. No registration floods against it, no callback forgery, and existing HTTP clients, load balancers, proxies and monitoring work unchanged.

03

The protocol

Discovery, registration, stages, the request, the patch, grants, recipients, extension data and failure.

Discovery

The scanner advertises its needs

GET /.well-known/mta-hooks

```
{
  "version": "1.0",
  "registration": "/v1/registrations",
  "serialization": ["application/cbor", "application/json"],
  "stages": ["connect", "rcpt", "data", "route", "delivery"],
  "properties": ["/session", "/envelope", "/message",
                 "/senderAuth", "/queue", "/ext/org.example.av"],
  "grants": [
    {"path": "/action",
     "values": ["accept", "quarantine", "reject", "discard"]},
    {"path": "/message/headers"},
    {"path": "/envelope/to/*/route"}
  ],
  "maxMessageSize": 52428800,
  "timeoutMs": 2000
}
```

Optional

The MTA may be configured with the registration URL instead.

A wish list

Stages, properties and grants the scanner would like. The MTA operator decides what it actually gets.

Registration

The MTA states the contract

POST /v1/registrations, label "spam"

```
"subscriptions": [
  {"stages": ["data"],
   "filter": {"authenticated": false},
   "properties": ["/session", "/envelope", "/message"],
   "grants": [{"path": "/action"}, {"path": "/response"},
              {"path": "/message/headers"},
              {"path": "/ext/org.example.spam", "max": 4096}],
   "onFailure": "continue"},
  {"stages": ["route"],
   "filter": {"envelopeTo": "*@partner.example"},
   "properties": ["/envelope", "/queue", "/ext/org.example.spam"],
   "grants": [{"path": "/envelope/to/*/route/name",
               "values": ["vpn-partner"]}],
   "onFailure": "tempfail"}
]
```

Written by the MTA

From its own configuration: what it will send and what it will allow.

Narrow, never widen

The scanner may drop properties and grants it does not need, or refuse with 422.

Subscriptions

Different traffic, different contract

First match wins

For each stage the MTA uses the first subscription that lists it and whose filter matches. No match, no call.

Each subscription carries its own properties, grants and failure policy.

Filters reuse JMAP

The Email filter conditions of RFC 8621, plus connection conditions:

`envelopeFrom` `envelopeTo` `clientIp` `serverPort` `tls` `authenticated` `authLogin`

At `route` and `delivery`, subscriptions match per recipient. Each one's grants and failure policy apply only to the recipients it matched.

Stages

Nine stages, one message

`connect` `ehlo` `mail` `rcpt` `data` `route` `transmit` `delivery` `dsn`

Inbound: an action and an SMTP reply

Outbound: the status of each recipient

State accumulates

A scanner subscribed only to `data` gets connection, TLS, authentication, envelope and message in one request, and can still reject at SMTP time.

One queue identity

The queue id is the same from `data` to the last `delivery` or `dsn`, so a scanner can keep what it learned at reception.

Stages

Two stages with no Milter equivalent

route

Before every delivery attempt

Runs each time recipients leave the queue, before the next hop is resolved. Sets a route per recipient.

Recipients with different routes never share an SMTP transaction.

transmit

Before every SMTP transaction

Runs once the transaction's envelope is final, with the exact reverse-path and recipients that will be sent.

Changes apply to this copy only and are not kept in the queue.

Hook request

The state, as the MTA holds it


```
{
  "stage": "data",
  "action": "accept",
  "decidedBy": "mta",
  "response": {"code": 250, "message": "Queued as 4F2A31B0C9"},
  "timeoutMs": 4200,
  "session": {
    "client": {"ip": "192.0.2.100", "ptr": "mail.example.org"},
    "tls": {"version": "TLSv1.3"},
    "counters": {"commands": 5, "pregreetBytes": 0}
  },
  "envelope": {
    "from": {"address": "sender@example.org"},
    "to": [{"address": "bob@example.com", "status": "accepted"}]
  },
  "message": {"size": 2048, "subject": "Meeting Tomorrow"},
  "senderAuth": {"spfMailFrom": "pass", "dmarc": "pass"}
}
```


Contents

Only the properties the subscription lists. The message as a JMAP Email object, raw bytes, or both.

HTTP header fields

- `MTA-Hooks-Registration`
- `MTA-Hooks-Request-Id`
- `MTA-Hooks-Context`

Hook response

The answer is a JSON Patch

Reject a message

```
[
  {"op": "replace", "path": "/action", "value": "reject"},
  {"op": "replace", "path": "/response",
   "value": {"code": 550, "enhancedCode": "5.7.1",
             "message": "Message rejected: virus detected"}}
]
```

Or nothing

`204`, or an empty patch, means no changes.

Blind writes

The patch applies to the state the MTA holds, so a scanner can add a header without having received the message.

Add a header field, and pass a score to later scanners

```
[{"op": "add", "path": "/message/headers/0",
  "value": {"name": "X-Spam-Status", "value": " No, score=0.4"}},
 {"op": "add", "path": "/ext/org.example.spam",
  "value": {"score": 0.4, "symbols": ["DKIM_VALID"]}}]
```

Validation

The MTA compares states, not operations

1

It applies

Per RFC 6902, including every `test`.

2

It is granted

Every location that differs is covered, within its values and max.

3

It is stricter

Ordered properties only tighten, unless relaxed.

4

It is valid

Types, stage rules, headers, addresses and MIME.

Applied to a copy, all or nothing. An invalid patch is discarded whole and counts as a scanner failure.

Grants

A grant is a path, with limits

```
[
  {"path": "/message/headers"},
  {"path": "/action", "relax": true},
  {"path": "/envelope/to/*/route/name",
   "values": ["default", "vpn-*"]},
  {"path": "/response/delayMs", "max": 30000},
  {"path": "/ext/org.example.spam", "max": 4096}
]
```

- **path**: covers everything below it; `*` matches any array index
- **values**: allowed strings, with wildcards
- **max**: a number ceiling, or a size in octets
- **relax**: may loosen ordered properties

No grant, no change. When grants overlap, the most specific path applies, so a narrow grant can restrict part of a broad one.

Grants

Restrict by default

Ordered from permissive to strict

Property	Order
`/action`	accept, quarantine, then reject, discard, disconnect
`/envelope/to/*/route/tls`	none, opportunistic, required, verified

Without relax

A scanner may keep the value or move it toward strict. It can reject what the MTA accepted, never accept what the MTA rejected.

With relax, the scanner becomes the final authority for that property: it can override a rejection, or lower TLS below the destination's policy, like `TLS-Required: No`. The operator grants it explicitly, and REQUIRETLS still wins.

Chains

Inbound: actions, in a chain

Action	Effect
accept	Continue normally
quarantine	Accept, then hold instead of delivering
reject	Send `/response`, a 4xx or 5xx
discard	Accept and drop
disconnect	Send `/response` and close

Ordered, attributed

Scanners run in configured order, each seeing the state left by the previous ones.

`decidedBy` names the label of whoever set the action, or `mta`.

Reject, discard and disconnect are terminal: the chain stops. At `rcpt` they apply only to the current recipient.

Recipients

Outbound: the status of each recipient

One status set, every stage

`pending` `accepted` `deferred` `failed` `delivered` `discarded`

Recipients are never removed. Dropping one means setting it to `discarded`, with no delivery and no DSN.

Stage	From	To
data	accepted	discarded
route, transmit, delivery	accepted, deferred	deferred, failed, discarded
delivery	failed	discarded
dsn	accepted	discarded

A scanner that fails or defers a recipient may set the reply the DSN reports, and reschedule the next attempt.

Extension data

A new field without a core patch

Written at data, read again at route

```
"ext": {
  "org.example.spam": {"score": 0.4},
  "org.example.av": {"verdict": "clean"}
}
```

- Namespaces are reverse domain names owned by their definer
- Subscription properties decide who reads a namespace; grants decide who writes it
- Kept with the message for as long as it is in the MTA
- Unknown namespaces are ignored

An MTA and a scanner agree on a namespace bilaterally. No wire format change, no registry, no ecosystem rollout.

Failure

On scanner failure, the operator decides

A scanner fails when it times out, exhausts its retries, returns anything but 200 or 204, or sends an invalid patch. The state is left unchanged and the subscription's `onFailure` applies.

onFailure: continue

Fail open

Proceed as if the scanner had made no changes. The default.

onFailure: tempfail

Fail closed

Inbound, answer 451 or 421. Outbound, keep the recipients deferred and retry later.

Each request carries its own `timeoutMs` budget, and a `MTA-Hooks-Request-Id` that stays the same on every retry, so scanners can deduplicate.

04

Use cases

Use cases, end to end, using only what the draft specifies.

Use case

Spam filtering at SMTP time

Subscribe

Only `data`, filtered to `{"authenticated": false}`. One call per message, with the whole session.

Decide

Sender authentication is already in the request. No recomputing SPF, DKIM or DMARC.

Answer

Reject with a 550 while the client is still connected, or quarantine, or accept with a header.

```
[{"op": "replace", "path": "/action", "value": "reject"},
 {"op": "replace", "path": "/response",
  "value": {"code": 550, "enhancedCode": "5.7.1", "message": "Spam"}}]
```

Use case

Slowing down a suspicious client

At connect: hold the greeting

```
[{"op": "add", "path": "/response/delayMs",
  "value": 15000}]
```

Signals in the session

`pregreetBytes` `pipeliningViolations` `bareLinefeeds` `invalidCommands` `firstCommandDelayMs`

Capped by the grant

`{"path": "/response/delayMs", "max": 30000}` keeps delays below SMTP client timeouts.

Withhold capabilities

At `ehlo`, offer AUTH only on some networks, or drop PIPELINING and CHUNKING for suspicious clients.

Use case

Mandatory routing, fail closed

Route through the VPN, unless another scanner already routed it

```
[{"op": "test", "path": "/envelope/to/0/route/name",
  "value": "default"},
 {"op": "replace", "path": "/envelope/to/0/route/name",
  "value": "vpn-partner"}]
```

The subscription

Filter `*@partner.example`, grant only the value `vpn-partner`, `onFailure: tempfail`.

Blast radius

If the scanner is down, only partner mail waits in the queue. Everything else takes default routing.

Failover

`route` runs before every attempt, with `attempts` and `lastAttempt`, so the scanner can switch paths after failures.

Use case

No bounce for noreply

At delivery

Drop the failed recipient

Setting a failed recipient to `discarded` suppresses its DSN. The scanner records the hard bounce in its own system instead.

At dsn

Or drop the DSN itself

Discarding the recipient of a DSN suppresses that DSN, for example backscatter control on a list.

```
[{"op": "replace", "path": "/envelope/to/2/status", "value": "discarded"}]
```

Per message and per recipient, through a grant, instead of a global switch.

Use case

Scanner to scanner, without X- headers

data

Spam scanner

Writes `/ext/org.example.spam`, granted with a 4096 octet max.

data

Next in the chain

Receives the namespace as a property, along with `decidedBy` on the action.

route

Days later

The routing scanner still sees `{"score": 0.4}` on the queued message.

Structured, attributed, size-capped, and never part of the mail that travels onward.

Use case

DKIM2 signing at transmit

The exact envelope

`transmit` runs per SMTP transaction, after the job is split, with the reverse-path and recipients of this copy. The signature covers exactly those recipients, never a guess.

Fresh every attempt

Changes are not kept in the queue. The stage runs again on retry, so each signature carries the time and envelope of the attempt that uses it.

DSNs too. At `dsn` the request describes the notification about to be sent, so the same scanner signs it the same way.

No per-MTA adapter: a signer like dkim2d can be one MTA Hooks scanner.

Use case

Delivery outcomes as data

Archiving

Capture at `data` with `tempfail`, so nothing is delivered unarchived. Proof of delivery arrives under the same queue id.

Bounce handling

The next hop's real reply per recipient, instead of VERP addresses and a bounce mailbox.

TLS reporting

Each attempt reports remote host, TLS version, policy and failure as RFC 8460 values.

Mapping back

Each problem and its answer

The problem	What answers it
Nothing after acceptance	`route`, `transmit`, `delivery` and `dsn`, in the same protocol
Routing gets one string	The full state at `route`, a granted route per recipient
Context by macro, silently absent	A registration that states exactly what is sent
A new field costs a core patch	Reverse-domain `/ext` namespaces, ignored if unknown
Filters talk through headers	Shared `/ext` data and `decidedBy` attribution
DKIM2 signatures and DSNs	`transmit` per transaction, `dsn` before sending
Operators cannot write filters	An HTTP endpoint that takes JSON and returns a patch

05

The case for a standard

The value is one scanner that works with every MTA.

Ecosystem

Write the scanner once

Today

One adapter per MTA

Milter for Postfix, local_scan for Exim, plugins for Haraka, Lua for KumoMTA. Every scanner rebuilds the same bridge, per server.

With a standard

One protocol, any MTA

A scanner discovers what a given MTA implements, and works with all of them.

Adoption is additive: no flag day, existing MTAs and filters unchanged. Gateways in both directions, Milter to MTA Hooks and back, can be built outside the WG.

Ecosystem

Built on existing standards

Transport

HTTP

TLS, connection reuse, load balancers, auth.

Encoding

JSON / CBOR

Chosen at registration.

Changes

RFC 6902

JSON Patch, with JSON Pointer paths.

Message model

JMAP

The Email object and its filters.

TLS outcomes

RFC 8460

Policy and failure values.

The draft specifies hook semantics and little else.

Status

Implementations and support

Implementations

- **Stalwart**: an earlier revision, in production
- **Rspamd**: stated intent to implement
- **Haraka**: offered on the list
- Independent scanners on GitHub, written by people who are not the draft authors

Support without a commitment

Heinlein Group, Beonex, KTP Digital, the SAKE project.

Prior sessions

MAILMAINT at 123, DISPATCH at 125, a side meeting at 126.

-02 folds in detailed proposals from mailbox.org operators on routing, observability and inbound signals.

06

The ask

Form a working group with a narrow charter.

Draft charter

In scope

Wire protocol

Requests, responses, headers, errors.

Discovery and registration

Well-known endpoint and the MTA to scanner contract.

Stages

Inbound and outbound, through delivery and DSN.

Routing delegation

Per recipient, job splitting, TLS policy.

Modification and chains

Changing state, ordering, termination.

Failure and security

Time budgets, fail open or closed, limits on what a scanner may change.

Draft charter

Out of scope, deliverables, milestones

Out of scope

- Specifying Milter, or a Milter gateway
- Mailbox-level filtering, which is Sieve
- Filtering rules, scores, detection logic
- Changes to SMTP, JMAP, Sieve or any other IETF protocol
- Scanner products, vendor extensions, policy languages
- Quarantine and admin user interfaces

Deliverables

- **Protocol specification**, Standards Track, based on draft-degennaro-mta-hooks
- **Operational considerations**, in the same document or a separate Informational one

Milestones

M+0 WG formed, draft adopted · **M+3** decide whether operational considerations are a separate draft · **M+12** WGLC on the protocol · **M+15** protocol to the IESG, WGLC on a separate ops draft · **M+18** ops draft to the IESG

BoF questions

Questions for the BoF

The problem and the charter

- Is the problem well understood, solvable, and useful to solve?
- Is the IETF the right place to do this work?
- Are the scope and deliverables of the draft charter correct?
- Should a working group be formed on that basis?

Commitment to the work

- Will you review documents?
- Will you edit one?
- Will you implement, in an MTA or a scanner?
- Would you deploy it?

Discussion: mtahooks@ietf.org



Over to the room

Is this a problem the IETF should take on, and are there enough of us to do the work?